StreamMLOps: Operationalizing Online Learning for Big Data Streaming & Real-Time Applications

Mariam BARRY
Institut Polytechnique de Paris
BNP Paribas
France
mariam.barry@bnpparibas.com

Jacob MONTIEL
University of Waikato
Amazon
United States of America
jacob.montiel@gmail.com

Albert BIFET
LTCI, Télécom Paris, IP Paris,
University of Waikato
New Zealand
abifet@waikato.ac.nz

Sameer WADKAR
Nikolay MANCHEV
Domino Data Lab
USA & United Kingdom
fds@dominodatalab.com

Max HALFORD
River Maintainer
Carbonfact
France

Raja CHIKY
Saad EL JAOUHARI*
ISEP- 3DS OUTSCALE*
France

Katherine B SHAKMAN
Joudi AL FEHAILY
Lyra & BNP Paribas
USA & France

Fabrice LE DEIT
Vinh-Thuy TRAN
Eric GUERIZEC
BNP Paribas

2023 IEEE 39th International Conference on Data Engineering (ICDE) | 979-8-3503-2227-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICDE55515.2023.00272

***Abstract*—Continuously learning and serving from evolving streaming data and serving in real-time is a challenging problem. Traditionally, data is partitioned and processed in batches to train machine learning (ML) models. In industrial applications, static models' performance drops over time (model degradation, concept drift), requiring new models to be trained with recent data and redeployed in production. The scientific community has been studying online and adaptive methods to address batch-learning limitations and continuously train AI tasks for industrial applications such as cyber-security, AIOps, anomaly scoring, and drift detection in stock markets. This paper deals with the MLOps aspects of deploying such online and dynamic models to address the requirements in the production systems for real-time applications. Our architectures - based on open-source tools such as Kafka and River - demonstrated how online learning methods could be scaled horizontally in production to meet the demands of a high-velocity streaming pipeline. We demonstrate an MLOps strategy to perform incremental learning from streaming data and continuously deploy the online learning model without pausing the inference pipeline. Indeed, the design satisfies requirements such as model versioning, monitoring, audibility and reproducibility of prediction in both a supervised and semi-supervised setting. Our experiments - for malicious URLs detection task - performed on high-dimensional and feature-evolving streaming data (more than 3 million features) establish the effectiveness and efficiency of online learning models compared to batch (static) machine learning regarding both time and space complexity. Finally, we provide some best practices on data engineering for deploying online models to process a real-time feature stream in production environments. Code is publicly available for reproducibility.**

***Index Terms*—Online Learning, Serving, MLOps, Data Streaming, Production, Real-Time, AIOps**

I. INTRODUCTION

Data streaming learning [1], also known as online learning [2], refers to continuously learning from new data and making predictions in an online fashion. MLOps can be defined as the set of practices (data modelling, data engineering, data processing, DevOps) and tools that aim to deploy and maintain machine learning models in production on top of a scalable and automated environment. We refer to *MLOps* as the process of operationalizing and maintaining machine learning in production [3]. Most of the methods released by the scientific community or models used in industry neglect that streaming data are evolving in real-world scenarios, and feature distributions are evolving [4]. A majority of models or MLOps (Machine Learning Operations) platforms [5] typically make a strong assumption (either explicitly or implicitly) that the data distribution is stationary, therefore, ignoring the limits of batch ML in production [6]. This provides motivation to consider online machine learning models, which are dynamically updated after each processed observation, without the need to retrain frequently on batched data stored in memory. In industrial environments, where streaming data is produced with evolving dynamics, there is a real need for architectures that manage machine learning (ML) models in production [7], that adapt to shifting data distributions [4], that deal with complex real-time application scenarios, and are self-maintaining. This work is motivated by the following question:

Given application events produced in streaming, where both the number and the type of the features are continuously evolving (ex. heterogeneous logs), how can we train incrementally & serve dynamic models in production without pausing the inference pipeline for real-time constraints ?

The problem we address in this work is how to build a system to operationalize dynamic online models, which learn and predict continuously for data streams. Developing and deploying machine learning systems is relatively fast and cheap, but maintaining them over time is difficult and expensive [8]. The unpredictability of online learning algorithms, due to the dynamic and evolving nature of the model, exacerbates the technical and functional costs of MLOps under real-time constraints, aka *Continuous MLOps*.

Typically, streaming and online models from the scien-

2375-026X/23/$31.00 ©2023 IEEE
DOI 10.1109/ICDE55515.2023.00272

Authorized licensed use limited to: Lebanese American University. Downloaded on August 26,2025 at 19:12:20 UTC from IEEE Xplore. Restrictions apply.

tific community. They are specifically designed for research purposes and not tailored toward production use. There is no simple and straightforward way to use such research-oriented solutions in an online learning context, especially when certain production constraints must be put in place. The key challenge here is to make the model operationalization (MLOps) compliant with the constraints imposed by existing requirements for robustness, performance, and manageable execution cost.

Despite the fact that online learning models carry a significant promise in various domains such as big data [9], cybersecurity [10], IoT [11], predictive maintenance [12], banking [13], agriculture [14], telecommunication [15], cloud computing [16], and healthcare [17], the lack of proper mechanisms to deploy such models in production has stalled their adoption. The two main factors holding back adoption in the industry are the lack of available labels at prediction time and the risk involved with updating models, which serve critical production and real-time requests without guarantees. Another factor is the specific requirements of a complete and robust streaming architecture that provides data to the models in real-time and is key for handling predictions and model updates. Our work aims to address the aforementioned challenges and provide a practical solution to operationalize online learning within industrial settings.

Advantages of online learning (single pass, incremental) compared to batch learning (multiple passes): The traditional approach to machine learning (batch) assumes that data is (1) always available and (2) fully represents the learning task or problem, i.e. data distribution is static. However, there are cases where keeping all the data in memory is unfeasible and/or impractical. Single-pass and incremental learning methods are similar in the sense that they only see the data once. A single-pass method learns from observing only once single instances, mini-batches or large batches of data; however, they are not required to provide predictions during this process. On the other hand, incremental methods mainly focus on learning from single instances and are explicitly designed to provide predictions as they learn (update their internal model of the data as seen so far). Additionally, recent incremental methods are designed to handle changes in data distribution, i.e. they adapt to changes in the data that represent the learning task. Data availability, resource management, and model availability (capable of prediction during training) are important challenges that require specific designs for learning methods. In practice, new batch learning methods are continuously trained and replaced in streaming environments. This is fine for multiple real-world problems, especially if data is static or changes slowly over time (models are replaced when they start to underperform). However, batch learning methods are not designed for this scenario and can not be easily modified to work in an incremental fashion.

Our contributions are the following :

- **MLOps Challenges** for serving evolving models in production, particularly within the Banking Industry
- **IT Data Categorization** and applications of online learning in the industry:
- **Online MLOps** architecture to scale & learn incrementally from big data streaming in real-time (Figure 3)
- **StreamMLOps** Practical design to horizontally scale online learning models for supervised and unsupervised learning to satisfy production requirements (Figure 4)
- **Data engineering best practices** to deploy online learning models to consume high volume and high-velocity streams (Flink, Kafka, Operationalization)

Code and reproducibility This paper is accompanied by a fully reproducible example provided as a Docker container and available in our GitHub repository[1]. Operationalization with industrial data cannot be shared for privacy reasons.

textcolorblackIn this paper, we present an approach and architectural design to use online learning on a high-velocity dataset (**OnlineMLOps** Fig. 3). We can apply the design for supervised learning in a setup dealing with delayed labelling and production requirements. We demonstrate the effectiveness of the proposed solution. First, we simulate the real-world scenario where labels are received delayed (with respect to prediction time) on Kafka topics. We horizontally scale three online learning methods (tree-based and ensemble-based approaches) while preserving good performance as shown in results Table I. We also present **StreamMLOps** in Fig 4, a design to use MLOps strategies to execute and manage model versions with user-defined regularity without interrupting the streaming pipeline. We present how this architecture can be implemented while complying with the industry and production requirements like prediction consistency, model versioning, prediction/model version reproducibility and apply auditability for model versions by storing artefacts in the model registry.

The paper is organized as follows: we present related work and state of the art in II. Section III-D discusses the problem and the challenges for online learning and banking applications. Section IV presents our proposed architecture using River [18] for online (streaming) learning and discusses how the proposed architecture has been implemented within a bank for real-time applications. Our evaluation and results discussion is done in Section V. The results clearly demonstrate that online learning can scale with high-velocity streaming data. Finally, Section VI provides lessons learned and best practices for the operationalization of online learning models.

Paper application potential for data practitioners The identified group of users that benefit from the solution and Code in this paper are:

- Industry practitioners for serving online learning models for real-time applications and production requirements.
- Researchers or data practitioners interested in building new continuous learning models for industrial applications.

[1]https://github.com/dominodatalab/online-learning-streaming

Authorized licensed use limited to: Lebanese American University. Downloaded on August 26,2025 at 19:12:20 UTC from IEEE Xplore. Restrictions apply.

- Data engineers and architects for building scalable platforms to extract valuable information from big data streaming.
- Data scientists or R&D teams interested in building online machine learning models suited for real-time applications.

II. Related work : Online Learning and MLOps

Online machine learning [2] builds stream models, which are updated based on information mined from the most recent examples. Unlike classical batch, machine learning models, trained once and then deployed in production, online learning algorithms operate in a streaming setting. **StreamAI** [7] presents how to deal with the industrial challenges of Continual Learning Systems in Production and AI Industrialization at scale. It covers the stream learning setting and the issues raised by *online machine learning models* during the complete ML & MLOps journey (building, deployment, maintenance). This work aims to address some of the industrial challenges presented in [7] for practical online learning and serving.

A. Online Machine Learning for Data Streams

A set of open-source tools for mining big data streams are available on a variety of platforms. MOA [20], the most popular stream learning library in Java, contains an extensive set of methods for multiple tasks, including classification, regression, drift detection, clustering, anomaly detection, active learning, etc. SAMOA [21] provides distributed implementations of stream learning algorithms that run on top of stream processing engines such as Apache Storm, Apache S4, Apache Samza, and Apache Flink. StreamDM [22] is designed to work with Spark Streaming. Vowpal Wabbit[2] is a machine learning system which pushes the frontier of machine learning with techniques such as online, hashing, reductions, learning2search, active, and interactive learning. There is a specific focus on reinforcement learning with several contextual bandit algorithms implemented and the online nature lending to the problem well. River [18] is an open-source library resulting from the merge of the two most popular Python packages for online machine learning: creme [23] and scikit-multiflow [24]. River[3] is popular both in academia and industry as it leverages the popularity of the Python language and the richness of its ecosystem (classification, clustering regression, multi-output drift detection and models pipelines). All these tools, coming from research labs, are intended for data stream processing. In order to deploy them in a production environment, e.g., within a bank, we should be able to integrate them into a data engineering architecture.

B. Online learning vs batch machine learning

Online machine learning methods have been extensively studied in recent literature, including applications for data streams in various tasks: ensemble learning [25], feature transformation techniques [26], classification [25], clustering [27], neural network [28] or more global surveys ([29], [30]). Several evaluations and benchmarks have demonstrated the advantages of online learning compared to batch learning methods in terms of performance. [27] present a benchmark on online clustering algorithms and compares them with traditional batch methods in terms of performance, memory and time usage. Experiments demonstrate that although online methods require fewer resources, they can obtain similar accuracy. [31] present a system that implements streaming decision trees that adapt to stream changes. This model is superior to standard decision trees, as they are built using a snapshot of data and can not evolve. Additional experiments discussing the benefit of online learning algorithms compared to batch methods are covered in [1].

C. Continuous learning and ML serving systems in practice

Google discuss the technical debts of (batch) machine learning (ML) systems in [8] and the cost of maintaining them over time. Software companies have released a variety of MLOps frameworks and platforms to smooth the path to production. Some of the themes include Google's TensorFlow Serving [32], Amazon's Sage Maker [33], Facebook's "FBLearner Flow" [34] and recently Meta's Looper [35]. However, the majority of these platforms focus on classical batch ML (with an offline retraining stage) and do not deal with the continuous learning setting where the system needs to serve a dynamic model in real-time and consequently update it after each sample is processed. In this paper, we focus on addressing the challenges in production in an online setting, where the model needs to serve real-time predictions while being continuously updated as stream instances are processed.

D. MLOps: Machine Learning Operationalization

Recent works discuss MLOps in various aspects: [5], and [36] present an introduction and architecture; [37] discuss the continuous integration and delivery for automated deployment of models; [38], [39] and [40] discuss more practical aspects and challenges of MLOps; [41], and [42] present open platforms and tools for MLops. Different concepts behind the term MLOps are discussed in [43]: what it is, why it is useful, and how it is implemented. In [44], the authors provide a formal definition of MLOps and AI Software Sustainability and their key challenges. A functional ML platform with DevOps capability, using existing continuous integration (CI) or continuous delivery (CD) tools and Kubeflow, is described in [45]. Recently, [46] presents Edge MLOps, an automation framework for IoT applications. Despite the recent attention around MLOps issues and models serving, the majority of the work focuses on batch machine learning settings. In this setting, the model is static and needs to be retrained periodically on a batch of data. In contrast, real-world applications involving data streams require online learning techniques, where the model can be dynamically retrained and updated. On the other hand, online learning models that are continuously trained and updated need to be robust to concept drift and data distribution changes.

[2]https://github.com/VowpalWabbit/vowpal_wabbit
[3]https://github.com/online-ml/river

Authorized licensed use limited to: Lebanese American University. Downloaded on August 26,2025 at 19:12:20 UTC from IEEE Xplore. Restrictions apply.

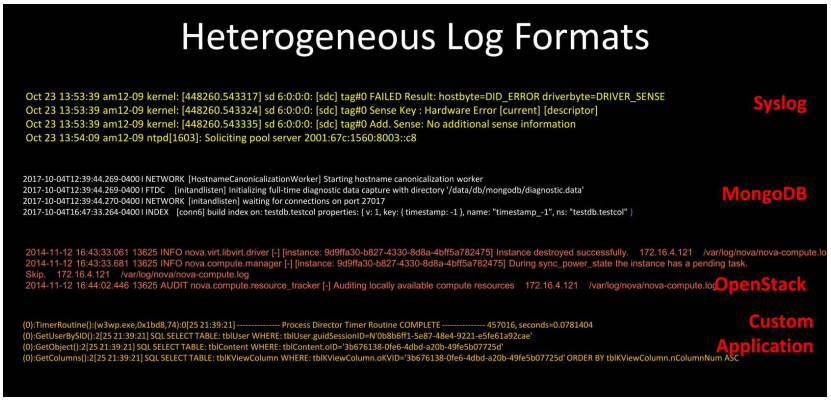

Figure 1: Illustration of heterogeneous log data produced by the industrial information system. Online learning and adaptive AI models are required to deal with the features-evolving data streams obtained after the parsing of Logs. Image source [19]

E. Comparison with the stream processing system and alternatives methods in related work

Here we discuss how our architectures and technology are used to compare to existing work in the area [47]–[52] (large-scale online learning systems, solutions based on stream processing platforms, algorithms for incremental learning, solutions for specific online machine learning problems).

Regarding online learning state of the art and scientific models, the rationale for using River [18] (instead of other tools mentioned in the related work) follows that Python is the most popular programming language among data scientists. Additionally, River and Python allow fast prototyping, which is a key factor during the iterative process of developing a machine learning-based solution. We agree with the reviewer that better performance can be achieved with other languages, such as Java. However, we believe that the proposed architecture is flexible enough to allow a more performance-oriented implementation if necessary. Regarding the stream processing art, in the paper experimentation, we have used cloud-based Kafka [53]. Streaming SQL pipelines are a good way to handle streaming features. Python is now becoming a defacto language for ML Model development. To adapt to this change, Streaming SQL pipelines do not support Python-based UDF. PySpark now supports UDFs in Python. This allows such systems to use the best of both worlds - Java for Feature Engineering and Python for model inference. An important aspect of turning around models fast is to ensure that the system the data scientist works in, is the same system that is deployed. Such polyglot systems (Java and Python) support the best-of-breed option where data engineers and data scientists can use the tooling and language of their choice. Kafka [53] is the most popular system for distributed messaging. It is fast and resilient and can scale out. However, other systems like AWS Kinesis or open-source products like Pulsar are acceptable. We simply chose Kafka for our experimentation because it can be deployed on-premise or on any cloud vendor platform. Furthermore, Kafka is a scalable platform for stream processing. Often, such systems need to scale out in response to load and back in when the load reduces. If the consumers of such systems are based on K8s, Keda is a system that allows scaling in the collection to application-specific metrics.

To the best of our knowledge, this work is the first to deal with the industrial challenges of serving online, incremental/continuous learning approaches with a dynamically updated model in a production setting. Our proposed solution and design combine research oriented with online learning models (river [18]) realized by the academic community with big data stream processing tools (Kafka [53]). Therefore we did not find related work which evaluates the end-to-end system performance to compare with.

III. Problem Statement: Online Learning Operationalization from Streaming Data

Rolling out models that learn and serve predictions based on high-velocity data streams is a challenging problem that presents difficulties in a number of areas.

A. Learning and serving predictions in the context of Big Data

When training data is supplied in a high-velocity fashion, the model training phase must be sufficiently quick so that no data gets discarded. Moreover, storing large amounts of data and applying batch training is often prohibitive from a cost perspective (storage) and could also be unfeasible because the storing/re-training overhead may introduce a lag that propagates through the pipeline and compounds with each storage/re-training iteration. Online learning can alleviate the issues related to storing and training using mini-batches, but the model updates still need to be performed swiftly enough so that the updating phase does not get overwhelmed by the incoming high-velocity observations. Whatever an architecture is designed to tackle these challenges, it needs to factor in the following:

1) The architecture should be able to address the challenges for both the training and model serving phases of the pipeline
2) The architecture should be elastically scalable so that it can be quickly adapted to changes in the predictions

Authorized licensed use limited to: Lebanese American University. Downloaded on August 26,2025 at 19:12:20 UTC from IEEE Xplore. Restrictions apply.

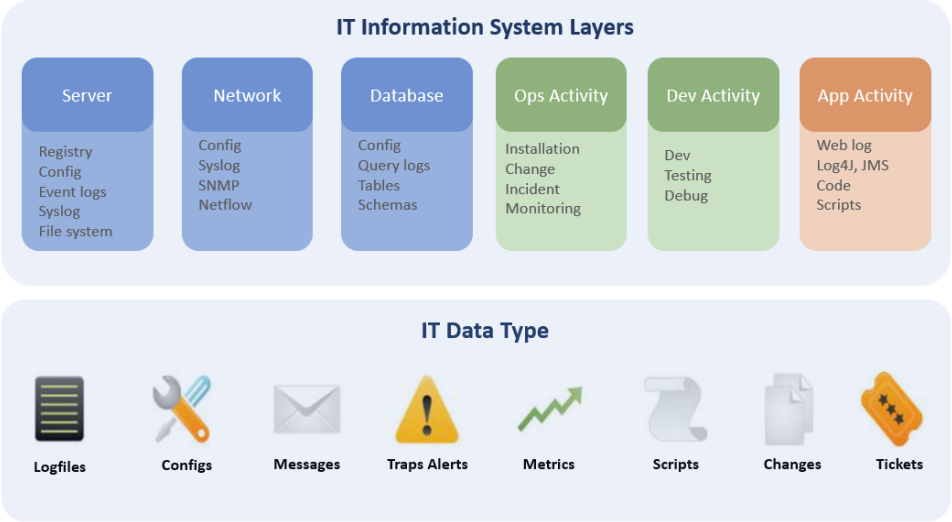

Figure 2: Categorization of different types of IT Data generated from large-scale information systems.

demand, data velocity, and volume. Ideally, it should allow for both vertical and horizontal scaling.

3) The architecture should be highly available, especially if there is no intermittent storage for the incoming data, as every disruption will be potentially associated with data loss.
4) The architecture should provide a robust and format-flexible way to offload predictions to the final consumers (i.e. decision makers)
5) The architecture must enable the online learning model to exhibit a number of desirable characteristics [1] including one pass over the data, constant or sub-linear complexity, efficient resource consumption (CPU and memory), parameter-free learning, and adaptability to concept drift
6) The architecture should be fully compliant with industry regulations in terms of Reproducibility, audit compliance, and traceability.

B. Requirements for operationalization of machine learning models in the banking industry

Here we expand on item 6 above. Using machine learning models in a highly regulated industry such as Finance comes with an additional set of requirements and safeguards that must be put around such critical systems.

Reproducibility of predictions and model This is required for the purposes of auditability. Every prediction generated by the model at time X must be fully reproducible. There should be a mechanism in place, which preserves model versions, so model instances can be resurrected as of time X and used for reproducible predictions that were generated in the past.

Consistency of prediction Given the same input features, the model should always produce the same outputs. This is challenging, especially if identical requests for scoring land on different model-serving instances.

Auditability / Transparency: Model predictions must be explainable. Moreover, there must be a mechanism in place that, when enabled, can log requests and model responses on demand.

Monitoring and roll-back to the previous model in production Models, especially in the case when they are being continuously updated, should be constantly monitored for model performance and concept drift. There should be mechanisms in place that allow for a streamlined switch back to a previous version if performance degradation is ever detected.

C. Motivations for online learning in the banking industry

BNP Paribas [4] is the leading bank in the Eurozone and a key player in international banking, serving millions of customers - individuals, corporations or institutions - in more than 60 countries. It operates a myriad of banking applications, continuously generating large quantities of data through interconnected streams (transactions, customer data, applications events, time series, logs, etc.) Large-scale information systems produce high-velocity streaming data with heterogeneous formats from multiple sources. Such data streams need to be processed by online machine-learning models that learn and self-update incrementally. This is important when streaming data are continuously evolving with change and drift occurrence. Some applications include critical use cases such as real-time fraud detection, network cyber-attack detection, weak signal detection, infrastructure monitoring and AIOps applications (Artificial Operations for IT Operations).

R&D (Research and Development) teams identified the need for a flexible and generic streaming platform to quickly deploy advanced AI methods and online learning models in production and extract valuable insights from real-time applications. This turned out to be a complex process that had to resolve various challenges and constraints, exacerbated by different consumer profiles during the industrialization journey (e.g. data scientists, data engineers, MLOps and R&D team)

[4]https://group.bnpparibas/

Authorized licensed use limited to: Lebanese American University. Downloaded on August 26,2025 at 19:12:20 UTC from IEEE Xplore. Restrictions apply.

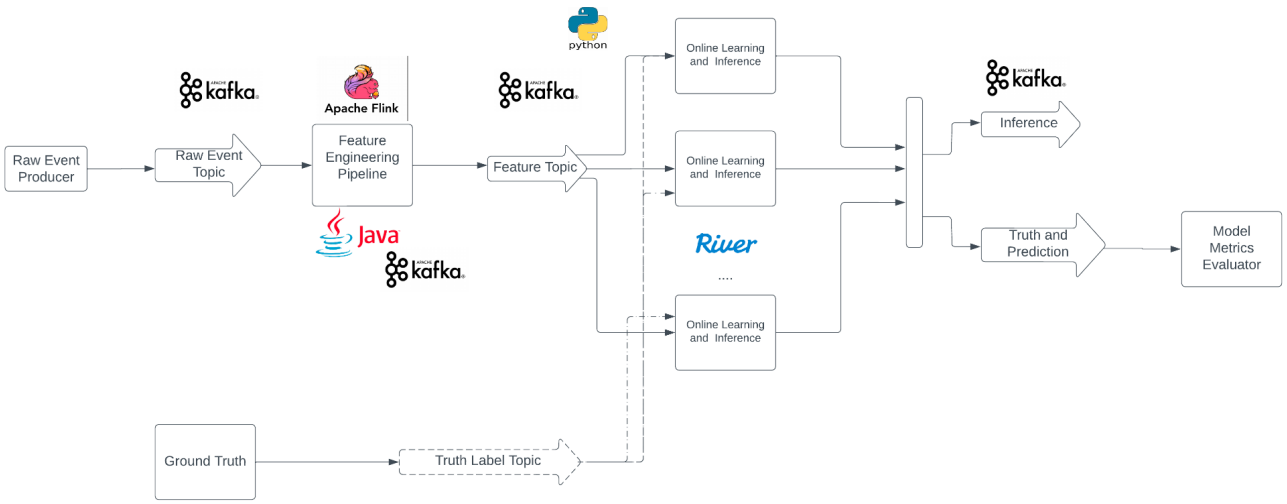

Figure 3: OnlineMLOps: Proposed Streaming Architecture for Online Learning and Online Inference (Horizontal scaling)

IT Data: Heterogeneous data streams from large-scale information system with dynamic and evolving behaviours

One of the motivations for using stream learning in the banking industry is to apply dynamic online models for evolving data streams produced by a wide range of information systems. In Figure 2, we illustrate a categorization of IT data, which are produced in a streaming fashion by various real-time IT or business applications. This heterogeneous IT data exhibits challenging properties, such as evolving distribution of the features but can be addressed by batch and static machine learning models. In the context of online learning, however, such properties present substantial obstacles. Therefore, in our evaluation, we use a highly-dimensional dataset whose characteristics are similar to heterogeneous banking IT data and application log events.

D. Case-Study: Online detection of malicious events from evolving data streams

A typical type of model that is used in the banking industry is "Anomaly Detection". Two common examples are Credit Card Fraud and Log-based IT Security Analysis. Such datasets are usually highly dimensional. For our experiments, we chose the Malicious URL [54] use-case because it has over 3 million features, and each record has a small subset of any of these features (similar to logs analysis). High-dimensional models are memory intensive and take a long time to train. We chose this challenging dataset because it is representative of challenges one would encounter in a real-world banking industry use case. Originally, [54] explored online learning approaches for detecting malicious Web sites (those involved in criminal scams) using lexical and host-based features of the associated URLs. The authors show that this application is particularly appropriate for online algorithms as the size of the training data is larger than what can be efficiently processed in batch. Moreover, the distribution of features that typify malicious URLs is continuously changing.

IV. Practical solution for Online Learning from streaming data (StreamMLOps)

In this section, we present our proposed architecture, which enables a production-grade operationalization for training and serving predictions from the online learning models.

A. Open source tools underpinning the architecture

The following open-source tools are fundamental building blocks of the proposed architecture: **Apache Kafka**[5] [53] a state-of-the-art open-source distributed event streaming platform; **Apache Flink**[6] [55] a framework and distributed processing engine for stateful computations over unbounded and bounded data streams; **mlflow**[7] for model registry; **XRAY** for experiments parallelization; **River** for online and incremental learning.

B. OnlineMLops: Architecture for Online Learning & Online Inference in Practice

Online learning refers to training the model as the ground truth arrives (based on the stored features). We will examine an architecture which uses Online Learning for Malicious URL Detection models. Such models have to respond very fast to incoming URLs and make a binary classification regarding whether a URL is malicious or not.

The basic design of online learning, when applied to unsupervised learning for a use-case such as malicious URL detection, is :

1) Features arrive on a potentially high-velocity stream such as Kafka topic
2) Parallel consumers of this stream host the model artefact, which makes the prediction and sends the response to another stream

[5]https://kafka.apache.org/
[6]https://flink.apache.org/flink-architecture.html
[7]https://mlflow.org/

Authorized licensed use limited to: Lebanese American University. Downloaded on August 26,2025 at 19:12:20 UTC from IEEE Xplore. Restrictions apply.

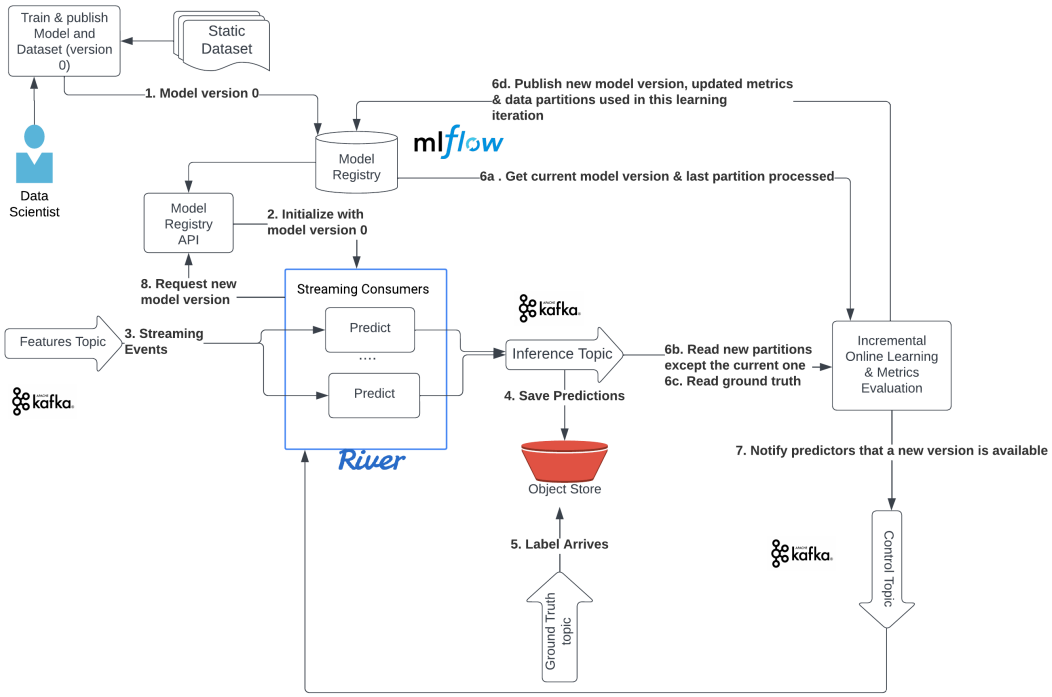

Figure 4: StreamMLOps: Architecture of Practical Online Learning Streaming for Production and Banking

3) After making the prediction, the same consumer will refine the model using the same set of features it used to make the prediction

Because a system based on Kafka is fault-tolerant and a consumer operator instance can fail and restart on another node, it is an excellent design to periodically store each consumer's model indexed by a consumer identifier. In case of failure, the recovered consumer operator can start using the last saved version of the model.

*a) **Feature engineering for stream processing**:* Figure 3 shows the feature engineering pipeline deployed using a system like Apache Flink. Apache Flink consumes raw events published to Apache Kafka, a distributed streaming platform. For simplistic features where the entire feature set can be created based on a single event, we do not need Apache Flink. However, in most streaming prediction-based use cases, feature Engineering must operate on complex data events windows. Windowing of events can be temporal (Window of 10 mins which rolls every 1 minute), count-based (Window of 10 events which rolls every event) or something more complex and custom-built. Apache Flink is a mature open-source streaming platform which supports rich windowing semantics ideal for feature engineering pipelines deployed at massive scale and event velocity. The streaming and feature engineering pipeline responsible for windowing capabilities is usually Java-based because it needs to be high-performance.

These features arrive at prediction topics based on a feature aggregation process.

This model is now deployed to a feature engineering pipeline using Flink [55], which receives the incoming features and makes predictions. The model instances are deployed in parallel, listening to one or more unique Kafka partitions of the topic. The set of features arriving at a specific prediction consumer is consistent. Given the same URL, the prediction consumer processing will be the same.

*b) **Model registry for predictions traceability**:* A popular open-source Model Registry solution is MLflow [8]. By versioning the model, we can achieve inference consistency (based on the model version) and auditability, and we are now able to apply model explainability for each model version and predictions produced by the model version. Because we have stored the incremental datasets (or a reference to them) used to produce each version of the model, we can even start from ground zero and replicate the model versions. This design is suitable when operating in industries which have strict regulatory requirements. In a real-world scenario, a Data Scientist would work on the feature engineering pipeline applied to raw data events. This will generate a set of features associated with the ground truth. The Data Scientist would then use this data to create a model, evaluate the model metrics, and, if satisfied, publish it to a model registry. The information stored for each version of an online learning model are:

- Source Code or reference to the source code for model generation

[8]https://mlflow.org/

Authorized licensed use limited to: Lebanese American University. Downloaded on August 26,2025 at 19:12:20 UTC from IEEE Xplore. Restrictions apply.

- Model Artifact (Ex. Model pickle file for Python)
- Incremental dataset (or reference to it) used to train the model version
- Model metrics for monitoring (performance, drifts, etc.)

This model is now deployed to a feature engineering pipeline which receives the incoming features and makes predictions. The model instances are deployed in parallel, listening to one or more unique Kafka partitions of the topic. The set of features arriving at a specific prediction consumer is consistent. This means that given the same URL, the prediction consumer processing will be the same. This model is now deployed to a feature engineering pipeline which receives the incoming features and makes predictions.

*c) **Continuously learning using the truth labels stored in Kafka**:* The ground truth arrives later. The system is configured in a way that the ground truth for a URL also arrives at the same consumer who made the prediction. In this use case, the prediction consumer will store the prediction and the features for a given URL in a local or remote store (usually known as an online feature store). On arrival of the ground truth, this consumer retrieves the prediction and features and uses this information to perform Online Learning. Thus each consumer running in parallel will learn independently of the other. The prediction and the ground truth will now be used to refine the metrics per consumer (or model).

C. *StreamMLops: Architecture for Practical Online Learning Audit & Reproducibility in industry*

For some industrial and banking use-case, the OnlineMLOps architecture proposed to deploy the model in production needs to be extended with additional requirements. For a use-case such as malicious URL detection, this design is adequate. However, when online learning is used for the banking sector in highly regulated industries, for fraud detection, credit risk scoring and any automated decision making, the design must handle the following non-functional requirements such as reproducibility of prediction and model, consistency of prediction and explainability for audit purpose. To address these requirements, we present **StreamMLOps** in Figure 4 modified design for the more generalized supervised learning model type. The steps of our architecture during the model serving life-cycle are the following:

1) The data scientist works with a static dataset to create version 0 of the model and store it in the Model Registry. Also, the metrics and the link to the dataset used for model training are included for model version reproducibility.
2) The streaming consumer consuming features from the topic of the feature will download version 0.
3) The feature topic will stream features consumed by the model, and the predictions are published to the Inference Topic.
4) The Predictions are saved to a Key/Value store (ex., an Object Store like S3 or a database like Redis)
5) When the Label arrives, it can be added to the Key/Value Store alongside the predictions.
6) The label's arrival will trigger a message containing the prediction, features and the label. The current version of the model is incrementally trained, and the new model version is published in the model registry. Again, the revised model metrics and the incremental dataset (or a link to it) are stored in the model registry to enable model version reproducibility.
7) A message is placed on the control tropic indicating that a new model version is available.
8) The Prediction/Inference consumers will fetch the latest version of the model and continue making inferences. The pipeline does not pause when the model version is refreshed. Each inference also contains the model version number it was produced with for auditability.

.

D. *Experimentation Architecture for Online Learning used in the paper and model operationalization*

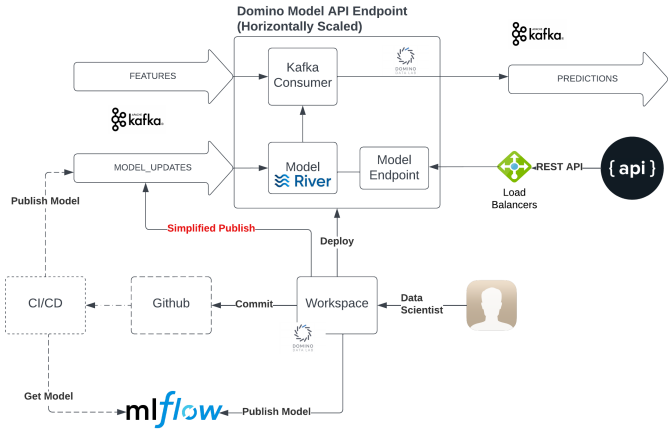

Figure 5: Streaming architecture used for the experiments for horizontal scaling of online learning. Code & Demo in Github

For the results published in the paper, we used the following experimentation setup and design illustrated in Figure 5. We use Domino Data Lab as our experimentation platform. Domino Data Lab is an MLOps platform enabling Data Scientists to manage the model development process from inception to production. The Data scientist will use the Domino workspace for interactive experimentation via a Jupyterlab notebook.

For the experiment execution, we used a simplified architecture where horizontally scaled model endpoint instances doubled up as Kafka Consumers as well as the API endpoint for interactive inferences. Domino also provides integration with Ray[9] as an alternative mechanism for processing Kafka topics. **Ray** provided the infrastructure to consume Kafka topics in parallel. Ray also provides a serving layer which can operate in push (prediction on event arrival) or pull (prediction on demand using an API) mode using a consistent API.

[9]www.ray.io

Authorized licensed use limited to: Lebanese American University. Downloaded on August 26,2025 at 19:12:20 UTC from IEEE Xplore. Restrictions apply.

To orchestrate our experiments and facilitate reproducibility, Domino platform will be used as a control plane to manage the end-to-end process for Online Learning to deployment. Others platforms and features can be used as the proposed StreamMLOps design is platform or product-agnostic.

a) ***Environment set up for experiments****:* For the streaming platform, we use the managed version of Apache Kafka. We start a horizontally replicated Domino Model Endpoint workspace with three cores and 27 GB of RAM. Domino workspace is used to run initial training, and experiment manager (managed MLflow in Domino Data Lab) is used as the model registry.

The experimentation methodology uses a Kafka topic (deployed to the free tier of Confluent Cloud) with partitions. A Domino Model Endpoint instance acts as a Kafka Consumer. It also provides a REST endpoint for making interactive API calls to the trained model. We obtained throughput requirements for the following three scenarios:

- 8 consumers (2 partitions per consumer) - Maximum parallelism
- 4 consumers (4 partitions per consumer) - Moderate parallelism
- 2 consumers (8 partitions per consumer) - Low parallelism

When the features arrive at a consumer, the model instance on that consumer performs Online Learning and Prediction using that feature set.

b) ***River: Online machine learning in Python****:* River [18] is an open-source online machine-learning library written in Python. The main focus of River is instance-incremental learning, meaning that every component (estimators, transformers, performance metrics, etc.) is designed to be updated one sample at a time[10]. This is in alignment with the streaming setting, where data is continuously generated, and once seen, it is no longer available in further iterations (no storage). The infrastructure for real-world applications is often difficult to change; accordingly, River is designed to be used on multiple architectures. It is not intended as a stand-alone tool; on the contrary, River is designed to integrate with other tools available in the Python ecosystem and in real-world systems. Another important design principle is that River is intended for researchers and practitioners. Complete learning experiments can be easily designed and implemented in a couple of lines of code. On the other hand, power users can easily extend and modify existing tools adapting them to their own needs. We choose River as the stream learning library in our system based on the above-mentioned qualities: flexibility to adapt to existing architectures and its potential to serve two kinds of users: data scientists and data engineers. In our proposed architecture, data is handled by the pipeline and passed to the River method, where the online model is exposed to the data stream, enabling online learning.

[10]River: Batch-incremental Learning is partially supported whereas instance-incrementally is fully supported.

c) Model operationalization using containers: The Domino [56] platform is implemented on top of Kubernetes [57], where it spins up containers on demand for running user workloads. The containers are based on Docker images, which are fully customizable, enabling data scientists to install additional libraries and frameworks, workspace tools (i.e., IDEs), and interactive applications (Dash, Flask, R Shiny, etc.) The containerized model is then stored as a Docker image in an internal registry. From there, the platform provides multiple paths to production — the model image can be hosted within the platform and exposed as a model serving instance (e.g., REST API), or it can be pushed to an external registry and operationalized in an infrastructure that is ring-fenced from the development environment (e.g., a separate on-premises Kubernetes cluster, AWS Sage Maker [33], etc.) The key advantages of this approach are: first, we have no discrepancy between development and production environments. Secondly, the industry (bank or any other domain) can use the preferred hosting infrastructure for models developed in Domino, making the work compatible with other services. Finally, it allows the industry to control an increasing proportion of deployment ops using internal CI/CD pipelines (Continuous Integration - Continuous Deployment).

V. Evaluation of OnlineMLOps for online learning and real-time inference

A. *Experimental framework for online training*

A series of experiments were conducted, with the main objective being the functional verification of the outlined streaming architecture. The secondary objective was a benchmarking and scalability exercise conducted with the goal of confirming the architecture's capability to handle high-velocity data.

a) ***Data set: high-dimensional and features-evolving streaming data (Malicious URL)****:* We used the dataset Malicious URL used by [54] to detect malicious Web sites from their URLs' lexical and host-based features. The dataset is available from River datasets [11] and is used for binary classification tasks. It is sparse and contains **2,396,130** samples and **3,231,961 features** where the number and distribution of the features are constantly evolving.

b) Metrics: We have computed the following metrics to evaluate the architecture in the table I.

The maximum **sustainable throughput** is the number of data elements (feature set) pushed to a Kafka topic per second. we measure the sustainable throughput per second per model. This throughput allows 8 consumers to keep up with the producers and the Latency stays constant for the entire run. As the number of consumers drops (to 4 or 2) the Latency increases. The typical time it takes for the learning method is represented by the **minimum Latency**. As the number of consumers drops to 4 and 2, the consumers fall behind and it takes longer for the features to reach the consumer. This increases the end-to-end Latency. In this paper latency is not

[11]http://www.sysnet.ucsd.edu/projects/url/url_svmlight.tar.gz

Authorized licensed use limited to: Lebanese American University. Downloaded on August 26,2025 at 19:12:20 UTC from IEEE Xplore. Restrictions apply.

Table I: We perform horizontal scaling tests on 3 stream learning models: Adaptive Random Forest (ARF), Streaming Random Patches with Naive Bayes (SRP–NB) as base model and Hoeffding Adaptive Tree (HAT). Top results per model are highlighted in bold and overall top results are underlined. Lower values are better for Latency and memory, larger values are better for F1-score.

	ARF - Adaptive R. Forest			**SRP–NB - Streaming RP**			**HAT - Hoeffding A. Tree**		
Kafka parallel consumers (models)	8	4	2	8	4	2	8	4	2
Kafka partitions per consumer	2	4	8	2	4	8	2	4	8
Total Samples Processed	100K	100K	100K	100K	100K	100K	100K	100K	100K
Samples processed per model	12.5K	25K	50K	12.5K	25K	50K	12.5K	25K	50K
Sustainable Throughput	32 768			2040			3072		
Latency									
Mean (sec)	**0.099**	4.122	36.759	**0.183**	19.210	49.348	**0.936**	36.949	167.663
Median (sec)	**0.092**	3.260	35.111	**0.125**	14.268	164.644	0.210	32.463	159.277
Max (sec)	**1.421**	18.908	106.903	**1.514**	91.624	418.789	15.556	115.113	415.217
Min (sec)	0.048	**0.047**	0.050	**0.053**	0.058	0.344	0.050	**0.049**	0.051
Memory									
Mean model size (MB)	**6.979**	9.080	9.021	**0.341**	0.358	0.386	**56.513**	61.807	77.175
Predictive performance									
Mean F1-score	0.815	**0.866**	0.827	0.777	0.819	**0.923**	0.822	0.824	**0.828**

the measure of how long it takes to train the model per feature set. It is the end-to-end Latency from when the feature set is written to a Kafka topic to when it was used to perform online learning.

For the online learning methods, we measure the predictive performance with the F1-score, a popular metric for binary classification data with class imbalance. The F1-score combines precision and recall into a single metric by taking their harmonic mean.

*c) **Online learning models trained for real-time model serving**:* The following online algorithms were tested as part of the functional and scalability experimentation.

- **Adaptive Random Forest** (ARF) [58] — An adaptation of the Random Forest algorithm for data streams which handles concept drift by updating the trees inside the ensemble.
- **Streaming Random Patches** (SRP) [59] — An incremental learning method that handles concept drift. The base model to build the ensemble can be any type of incremental classifier. Base models are trained on different partitions of data or "patches". In our experiments, we test with two base models for this method, Naive Bayes and Hoeffding Adaptive Tree.
- **Naive Bayes** (NB) [1] – A probabilistic classifier based on applying Bayes' theorem with strong independence assumptions between the features.
- **Hoeffding Adaptive Tree** (HAT) [60]. Builds a decision tree incrementally (Hoeffding Tree) and if concept drift is detected, the model starts creating alternative branches. If an alternate branch performs better in new data, then it replaces the original branch in the tree.

We select these methods to showcase the seamless deployment of machine learning models in the proposed architecture. The implementation of these algorithms comes directly from River. We use the default parameters reported in the official API of River[12] package for online machine learning.

B. Experiments results and scalability tests analysis

In Table I we present the results of our experiments performed on a subset of the Malicious dataset with 100,000 samples. For each model, we perform three experiments with horizontal scaling. The data stream is processed in 8, 4 and 2 Consumers and the data stream has 16 partitions (Kafka topics).The stream is equally partitioned at random over the number of instances on each experiment. Each partition contains an instance of the learning model performing training and prediction.

We assess operational performance by measuring Latency, memory and predictive performance. We show that horizontal scaling improves throughput and analyze how this impacts models and the overall system's performance. We measure model size on disk at regular intervals after each 1000 predictions of the stream and report the average value. For predictive performance, we report the average F1 score for the entire stream. From the results listed in Table I, we derive the following insights:

Predictive performance (F1-score) increases with the number of training samples which is expected as models receive more training data. When we increase the parallelism (horizontal scaling), the parallel models see fewer samples and reach lower F1 scores. The amount by which F1-score is impacted depends on the type of model used, SRP-NB being the one with the largest drop in performance. On the other hand, ARF and HAT show relatively marginal drops in F1-score. ARF is an interesting case since increasing parallelism from 2 to 4 instances results in a better F1 score, a possible explanation is the additional level of randomness introduced by the data partitioning. This result highlights the importance

[12]https://riverml.xyz/

Authorized licensed use limited to: Lebanese American University. Downloaded on August 26,2025 at 19:12:20 UTC from IEEE Xplore. Restrictions apply.

of properly tuning the level of parallelization to find the sweet spot between predictive performance and throughput.

Memory wise we observe that model size decreases with parallelization, albeit the variation is relatively small. This is explained by two factors, (1) we are effectively training more model instances, and (2) trained models are smaller as they receive fewer data. This is especially relevant to tree-based methods (ARF and HAT) that increase the depth of the tree during training (not indefinitely). Model size for SRP–NB is relatively small given the fact that NB is a simple model with a small memory footprint. It is important to notice that latency results need to be interpreted with caution as Latency can vary depending on the configuration of the stream producer and consumer.

C. Parallelism impact on Latency and sustainable throughput

We notice from the table I that the performance of the system for stream processing and learning depends entirely on the model and the fixing the number of partitions. If we increase the partitions and consumers proportionately, it increases. Ultimately it is bounded by model training time for online learning. It also depends on network bandwidth, types of instances used for Kafka and consumers of Kafka. The high maximum values reported for HAT can be for one of two reasons: 1) model takes longer to warm up and the first very records will have a higher max value, 2) The model consumers fall slightly behind for the last few records. These records arrive at the consumer a bit late and contribute to the high max value. This cause is due to Kafka system delays and not the online learning method. The numbers are more indicative of which models are faster to train and which ones are slower. The slower ones need more parallelism to get the same throughout. And hence more expensive infrastructure wise.

VI. Lessons learned and MLOps best practices

This paper explored several best practices for Online Learning on a high-velocity feature stream. We explored the best practices for Online Model Learning, Model Serving, the role of Model Registry, MLOps processes to deploy models on a live stream without requiring pipeline restarts, and, most importantly, handling the practical challenge of Online Learning when the ground truth arrives at a later time. Next, we share some lessons learned and best practices for production (MLOps).

A. Feature Engineering with Apache Flink

In the discussion above, we have assumed that engineered features arrive at a topic which the inference consumer uses to produce inferences. However, in a streaming architecture, feature engineering presents significant challenges.

For example, if one of the features is the MAX value within all values received within the last 30-minute window, which rolls every minute in a high-velocity stream. Given an us-case where we aim to detect credit card fraud, a feature could be the geodetic distance between two consecutive transaction locations for the last 30 transactions. It is impractical to save these features to an online feature store and retrieve them when making predictions. While extremely simple conceptually, such a system would be prohibitively expensive due to the large amount of horizontally scaled components needed to work under heavy load.

For an unsupervised learning or fraud use case, predictions must be made on each transaction. The features have to be constantly generated on windows of events which are stateful. Apache Flink provides an excellent system for such use cases. It has rich windowing and trigger semantics. The events can be stored reliably in the operator's local state (speed and low latency, and high throughput), its checkpointing capability allows it to recover from failures without losing window states, and its save point capabilities allow modifications and redeployment of the feature engineering pipeline without losing state. The streaming feature engineering and inference pipeline must be backed by a feature and model metadata store to enable a horizontally scalable system capable of live feature onboarding.

B. Architecture deployment in industry for production

a) ***Model serving****:* We are using model serving over the stream in our experimentation. We have a python based Kafka consumer which processes the features as they arrive and makes a prediction. The model is just a python program which can be served in any manner needed by the use case. And even multiple ways, such as on a stream and a model endpoint.

b) ***Automating Model updates and Auto-scaling*** *:* Auto-scaling can be done with Kubernetes. Model updates are propagated via a Model Registry. We propose MLflow, but virtually any model registry would work fine.

c) ***Architecture deployment on Cloud****:* We have used open-source systems for model training and deployment. These systems can easily be deployed to the cloud using Infrastructure or Software as a Service.

d) ***Dealing with supervised online learning and Delayed labelling****:* In the real world, the ground truth only arrives later. Therefore the architecture must be capable of storing the predictions and the feature set using a business key (Ex. Credit Card Number and Transaction Id) to enable Online Learning when the truth arrives. We have described this in our architecture diagram for production application in Figure 4.

C. Dealing with real-time applications using Kafka

Because listening from a Kafka Topic is an I/O intensive process, it is good to have each Model Serving Instance listen to (and write to) fewer partitions of the source and sink topic. At most, there can be as many parallel instances as the number of Kafka partitions on a source topic. Throughput is maximized if the number of partitions on the source and sink topics is identical. As the ratio of partitions to parallel Model Serving Instances drops (increasing parallelism), the

Authorized licensed use limited to: Lebanese American University. Downloaded on August 26,2025 at 19:12:20 UTC from IEEE Xplore. Restrictions apply.

throughput increases (and latency drops). We published thousands of records together to the Kafka topic for each published operation. Increasing the number may improve throughput but at the slight cost of increased latency. Using a too-high value, however, may exhaust memory. Ultimately the selection of this number is a trade-off between throughput and memory consumption because the messages are buffered in the memory of the Model Serving Operator before they are flushed to the inference topic. The best selection will depend on the latency, the payload size of the inference message, and the memory available on the machine hosting the Model Inference operator. In this study, uncompressed messages were published to the source and inference topics. Because this architecture is I/O intensive (as opposed to CPU intensive), the use of compression may increase throughput without hurting other performance metrics. It is recommended to test the impact of message compression for production use cases.

D. Architecture discussion

The benefit of the architecture is that it is open architecture. The various components can be replaced with similar platform components. Horizontally scalable systems can become expensive. In this paper, we used plain JSON for sending messages to Kafka. JSON is bulkier than binary formats like AVRO. We should use binary payloads instead of text-based payloads to make our system more cost-efficient. This will allow us to meet the throughput requirements with less hardware.

E. Some Business applications of Online ML and our solution

Here we provide examples of industrial use-cases that could benefit from our proposed solution and code source.

- Anomaly detection of critical streaming applications
- AIOps applications from heterogeneous IT data streams
- Real-time Fraud detection from high-velocity transactions
- Automated detection & categorization of Cyber-attacks
- Time series drifts detection from financial markets data
- Change detection in Telecommunication network traffic

Banking applications when online learning solves problems not addressed by alternatives: Systems like Credit Card Fraud Detection, Unsupervised Learning Algorithms. Or Spam detection will produce inferences. These systems will also have access to ground truth reasonably soon (A customer will confirm if the credit card transaction was a fraud or not). Such systems can benefit from online learning as soon as the ground truth becomes available (or even when an inference request is made for Unsupervised Learning). Furthermore, banks used DevOps and MLOps to simplify the process of deployment and monitoring of systems and models. Online Learning systems leverage all data instead of depending on sampling for model training. The proposed system supports high throughput and latency. The system also supports constant relearning of the model without compromising the important requirements of auditability and reproducibility. Banking systems must not only be fast but must also be responsible because the cost of incorrect predictions can be high. Our system emphasizes those features.

Code source We release all the materials (code and data) in GitHub repository[13] to reproduce experiments and speed up the deployment of architectures design in Fig. 4 and Fig. 5 by the research community and data practitioners.

CONCLUSION

This paper deals with the MLOps aspect of deploying dynamic online machine-learning models for streaming data. First, we provide key industrial challenges of online learning and the main blockers to their adoption in the banking sector. Second, we propose a reference architecture that allows continual learning and serving in a way that satisfies the imposed production constraints. Experiments on a high-dimensional dataset with millions of features prove that online learning methods, ensemble-based and adaptive approaches are efficient (in terms of time, space and accuracy) to deal with feature-evolving data streams. We demonstrate that the proposed system can successfully ingest and process high-velocity streaming data and that online learning models can be horizontally scaled for practical purposes. StreamMLOps design provides a seamless bridge between data science and data engineering in a way that satisfies the imposed production constraints. We also discuss critical issues around training and scoring, reproducibility, and traceability to deal with production and banking requirements.

We hope the insights shared here will encourage additional development of AI models that can be easily deployed and maintained in production. A hybrid research approach where researchers, data scientists, and engineers / MLOps collaborate within the same team should speed up production-oriented delivery for complex applications scenarios. Our work and resources can be used by industry practitioners to deploy online learning models, which are currently released, and dynamic and adaptive AI models to deliver real-time business applications. Researchers can use our work to develop and benchmark new or improved models to support the use cases for applying self-adaptive and automated learning and online models on a high-velocity stream with high throughput requirements. Future research could address some of the industrial challenges discussed in **StreamAI** [7], including continuously serving & maintaining self-learning AI models in production while dealing with constantly evolving laws and regulations constraints regarding data privacy, audit and automated decision (ethics in AI, Cloud act, GDPR 2, etc..).

ACKNOWLEDGEMENTS

We warmly acknowledge Yingmin LI, Jean-Luc BILLY, Brice SIMON, Ludovic AUTAA and Bassim Ahmed EL BAROUDI for their feedback on the technical challenges in production platforms. We thank the researchers from DIG (Data, Intelligence & Graph) team at Télécom Paris for their insightful discussions. This research was partially funded by BNP Paribas (IT Group, RH Group) and ANRT. Domino Data Lab provided the computing resources, environment, and technical expertise for orchestrating and running the experiments.

[13]https://github.com/dominodatalab/online-learning-streaming

Authorized licensed use limited to: Lebanese American University. Downloaded on August 26,2025 at 19:12:20 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] A. Bifet, R. Gavalda, G. Holmes, and B. Pfahringer, *Machine learning for data streams: with practical examples in MOA*. MIT press, 2018.

[2] S. Hoi, D. Sahoo, J. Lu, and P. Zhao, "Online learning: A comprehensive survey," 02 2018.

[3] Y. Zhao, "Machine learning in production: A literature," 2021.

[4] T. Diethe, T. Borchert, E. Thereska, B. Balle, and N. Lawrence, "Continual learning in practice," 2019. [Online]. Available: https://arxiv.org/abs/1903.05202

[5] D. Kreuzberger, N. Kühl, and S. Hirschl, "Machine learning operations (mlops): Overview, definition, and architecture," *arXiv preprint arXiv:2205.02302*, 2022.

[6] J. Klaise, A. Van Looveren, C. Cox, G. Vacanti, and A. Coca, "Monitoring and explainability of models in production," *arXiv preprint arXiv:2007.06299*, 2020.

[7] M. Barry, A. Bifet, and J.-L. Billy, "StreamAI: Dealing with Challenges of Online Learning Systems for Serving AI in Production," in *IEEE 45th International Conference on Software Engineering (ICSE), Software Engineering in Practice (SEIP)*, 2023.

[8] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," *Advances in neural information processing systems*, vol. 28, 2015.

[9] L. Rettig, M. Khayati, P. Cudré-Mauroux, and M. Piórkowski, "Online anomaly detection over big data streams," in *Applied data science*. Springer, 2019, pp. 289–312.

[10] G. Li, Y. Shen, P. Zhao, X. Lu, J. Liu, Y. Liu, and S. C. Hoi, "Detecting cyberattacks in industrial control systems using online learning algorithms," *Neurocomputing*, vol. 364, pp. 338–348, 2019.

[11] A. Bifet and J. Gama, "Iot data stream analytics," pp. 491–492, 2020.

[12] J. Carvajal Soto, F. Tavakolizadeh, and D. Gyulai, "An online machine learning framework for early detection of product failures in an industry 4.0 context," *International Journal of Computer Integrated Manufacturing*, vol. 32, no. 4-5, pp. 452–465, 2019.

[13] M. Barry, A. Bifet, R. Chiky, J. Montiel, and V.-T. Tran, "Challenges of machine learning for data streams in the banking industry," in *International Conference on Big Data Analytics*. Springer, 2021, pp. 106–118.

[14] J. C. Moso, S. Cormier, C. de Runz, H. Fouchal, and J. M. Wandeto, "Anomaly detection on data streams for smart agriculture," *Agriculture*, vol. 11, no. 11, p. 1083, 2021.

[15] S. Bhatia, A. Jain, P. Li, R. Kumar, and B. Hooi, "Mstream: Fast anomaly detection in multi-aspect streams," in *Proceedings of the Web Conference 2021*, 2021, pp. 3371–3382.

[16] A. Vervaet, "Monilog: An automated log-based anomaly detection system for cloud computing infrastructures," in *2021 IEEE 37th International Conference on Data Engineering (ICDE)*, 2021, pp. 2739–2743.

[17] C. S. Lee and A. Y. Lee, "Clinical applications of continual learning machine learning," *The Lancet Digital Health*, vol. 2, no. 6, pp. e279–e281, 2020.

[18] J. Montiel, M. Halford, S. M. Mastelini, G. Bolmier, R. Sourty, R. Vaysse, A. Zouitine, H. M. Gomes, J. Read, T. Abdessalem, and A. Bifet, "River: machine learning for streaming data in python," 2020.

[19] B. Debnath and W. Dennis, "Fast log analysis made easy by automatically parsing heterogeneous logs," 11 2017.

[20] A. Bifet, G. Holmes, R. Kirkby, and B. Pfahringer, "MOA: massive online analysis," *J. Mach. Learn. Res.*, vol. 11, pp. 1601–1604, 2010.

[21] G. D. F. Morales and A. Bifet, "Samoa: scalable advanced massive online analysis." *J. Mach. Learn. Res.*, vol. 16, no. 1, pp. 149–153, 2015.

[22] A. Bifet, S. Maniu, J. Qian, G. Tian, C. He, and W. Fan, "Streamdm: Advanced data mining in spark streaming," in *2015 IEEE International Conference on Data Mining Workshop (ICDMW)*. IEEE, 2015, pp. 1608–1611.

[23] M. Halford, G. Bolmier, R. Sourty, R. Vaysse, and A. Zouitine, "creme, a Python library for online machine learning," 2019. [Online]. Available: https://github.com/MaxHalford/creme

[24] J. Montiel, J. Read, A. Bifet, and T. Abdessalem, "Scikit-multiflow: A multi-output streaming framework," *Journal of Machine Learning Research*, vol. 19, no. 72, pp. 1–5, 2018. [Online]. Available: http://jmlr.org/papers/v19/18-251.html

[25] H. M. Gomes, J. P. Barddal, F. Enembreck, and A. Bifet, "A survey on ensemble learning for data stream classification," *ACM Computing Surveys (CSUR)*, vol. 50, no. 2, pp. 1–36, 2017.

[26] M. Bahri, A. Bifet, S. Maniu, and H. M. Gomes, "Survey on feature transformation techniques for data streams," in *Proceedings of the Twenty-Ninth International Conference on International Joint Conferences on Artificial Intelligence*, 2021, pp. 4796–4802.

[27] J. Montiel, H.-A. Ngo, M.-H. Le-Nguyen, and A. Bifet, "Online clustering: Algorithms, evaluation, metrics, applications and benchmarking," in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, ser. KDD '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 4808–4809. [Online]. Available: https://doi.org/10.1145/3534678.3542600

[28] Z. Hammami, M. Sayed-Mouchaweh, W. Mouelhi, and L. Ben Said, "Neural networks for online learning of non-stationary data streams: a review and application for smart grids flexibility improvement," *Artificial Intelligence Review*, vol. 53, no. 8, pp. 6111–6154, 2020.

[29] J. Gama, "A survey on learning from data streams: current and future trends," *Progress in Artificial Intelligence*, vol. 1, no. 1, pp. 45–55, 2012.

[30] H. M. Gomes, J. Read, A. Bifet, J. P. Barddal, and J. Gama, "Machine learning for streaming data: state of the art, challenges, and opportunities," *ACM SIGKDD Explorations Newsletter*, vol. 21, no. 2, pp. 6–22, 2019.

[31] A. Bifet, J. Zhang, W. Fan, C. He, J. Zhang, J. Qian, G. Holmes, and B. Pfahringer, "Extremely fast decision tree mining for evolving data streams," in *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2017, pp. 1733–1742.

[32] C. Olston, N. Fiedel, K. Gorovoy, J. Harmsen, L. Lao, F. Li, V. Rajashekhar, S. Ramesh, and J. Soyke, "Tensorflow-serving: Flexible, high-performance ml serving," 2017. [Online]. Available: https://arxiv.org/abs/1712.06139

[33] D. Nigenda, Z. Karnin, M. B. Zafar, R. Ramesha, A. Tan, M. Donini, and K. Kenthapadi, "Amazon sagemaker model monitor: A system for real-time insights into deployed machine learning models," *arXiv preprint arXiv:2111.13657*, 2021.

[34] J. Dunn, "Introducing fblearner flow: Facebook's ai backbone," https://engineering.fb.com/2016/05/09/core-data/introducing-fblearner-flow-facebook-s-ai-backbone/, accessed: 2022-08-20.

[35] I. L. Markov, H. Wang, N. S. Kasturi, S. Singh, M. R. Garrard, Y. Huang, S. W. C. Yuen, S. Tran, Z. Wang, I. Glotov, T. Gupta, P. Chen, B. Huang, X. Xie, M. Belkin, S. Uryasev, S. Howie, E. Bakshy, and N. Zhou, "Looper: An end-to-end ml platform for product decisions," in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, ser. KDD '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 3513–3523. [Online]. Available: https://doi.org/10.1145/3534678.3539059

[36] M. Treveil, N. Omont, C. Stenac, K. Lefevre, D. Phan, J. Zentici, A. Lavoillotte, M. Miyazaki, and L. Heidmann, *Introducing MLOps*. O'Reilly Media, 2020.

[37] S. Garg, P. Pundir, G. Rathee, P. Gupta, S. Garg, and S. Ahlawat, "On continuous integration/continuous delivery for automated deployment of machine learning models using mlops," in *2021 IEEE Fourth International Conference on Artificial Intelligence and Knowledge Engineering (AIKE)*. IEEE, 2021, pp. 25–28.

[38] T. Granlund, A. Kopponen, V. Stirbu, L. Myllyaho, and T. Mikkonen, "Mlops challenges in multi-organization setup: Experiences from two real-world cases," in *2021 IEEE/ACM 1st Workshop on AI Engineering-Software Engineering for AI (WAIN)*. IEEE, 2021, pp. 82–88.

[39] N. Gift and A. Deza, *Practical MLOps*. " O'Reilly Media, Inc.", 2021.

[40] S. Mäkinen, H. Skogström, E. Laaksonen, and T. Mikkonen, "Who needs mlops: What data scientists seek to accomplish and how can mlops help?" in *2021 IEEE/ACM 1st Workshop on AI Engineering-Software Engineering for AI (WAIN)*. IEEE, 2021, pp. 109–112.

[41] M. Lukas, D. Stock, and A. Csiszar, "Fabos: Towards an open, distributed, real-time-capable, and secure operating system for production," *Procedia CIRP*, vol. 104, pp. 962–967, 2021.

[42] G. Fursin, H. Guillou, and N. Essayan, "Codereef: an open platform for portable mlops, reusable automation actions and reproducible benchmarking," *arXiv preprint arXiv:2001.07935*, 2020.

[43] S. Alla and S. K. Adari, *What Is MLOps?* Berkeley, CA: Apress, 2021, pp. 79–124. [Online]. Available: https://doi.org/10.1007/978-1-4842-6549-9_3

Authorized licensed use limited to: Lebanese American University. Downloaded on August 26,2025 at 19:12:20 UTC from IEEE Xplore. Restrictions apply.

[44] D. A. Tamburri, "Sustainable mlops: Trends and challenges," in *2020 22nd International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNASC)*, 2020, pp. 17–23.
[45] Y. Zhou, Y. Yu, and B. Ding, "Towards mlops: A case study of ml pipeline platform," in *2020 International Conference on Artificial Intelligence and Computer Engineering (ICAICE)*, 2020, pp. 494–500.
[46] E. e. a. Raj, "Edge mlops: An automation framework for aiot applications," in *2021 IEEE International Conference on Cloud Engineering (IC2E)*.
[47] J. Karimov, T. Rabl, A. Katsifodimos, R. Samarev, H. Heiskanen, and V. Markl, "Benchmarking distributed stream data processing systems," in *2018 IEEE 34th International Conference on Data Engineering (ICDE)*. IEEE, 2018, pp. 1507–1518.
[48] D. Foroni, C. Axenie, S. Bortoli, M. Al Hajj Hassan, R. Acker, R. Tudoran, G. Brasche, and Y. Velegrakis, "Moira: A goal-oriented incremental machine learning approach to dynamic resource cost estimation in distributed stream processing systems," in *Proceedings of the International Workshop on Real-Time Business Intelligence and Analytics*, ser. BIRTE '18. New York, NY, USA: Association for Computing Machinery, 2018. [Online]. Available: https://doi.org/10.1145/3242153.3242160
[49] O.-C. Marcu, A. Costan, G. Antoniu, M. Pérez-Hernández, B. Nicolae, R. Tudoran, and S. Bortoli, "Kera: Scalable data ingestion for stream processing," in *2018 IEEE 38th International Conference on Distributed Computing Systems (ICDCS)*, 2018, pp. 1480–1485.
[50] C. Axenie, R. Tudoran, S. Bortoli, M. Al Hajj Hassan, D. Foroni, and G. Brasche, "Starlord: Sliding window temporal accumulate-retract learning for online reasoning on datastreams," in *2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA)*, 2018, pp. 1115–1122.
[51] A. Bifet, R. Gavalda, G. Holmes, and B. Pfahringer, *Machine learning for data streams: with practical examples in MOA*. MIT press, 2018.
[52] C. Salort Sanchez, R. Tudoran, M. Al Hajj Hassan, S. Bortoli, G. Brasche, J. Baumbach, and C. Axenie, "An online incremental clustering framework for real-time stream analytics," in *2019 18th IEEE International Conference On Machine Learning And Applications (ICMLA)*, 2019, pp. 1480–1485.
[53] Kafka, "Apache Kafka, an open-source distributed event streaming platform," https://kafka.apache.org/, 2022, [Online; accessed 5-May-2022].
[54] J. Ma, L. K. Saul, S. Savage, and G. M. Voelker, "Identifying suspicious urls: an application of large-scale online learning," in *Proceedings of the 26th annual international conference on machine learning*, 2009, pp. 681–688.
[55] Flink, "Apache Flink, a framework and distributed processing engine," https://flink.apache.org/, 2022, [Online; accessed 04-june-2022].
[56] "Domino data science platform," https://www.dominodatalab.com/product/domino-data-science-platform/, accessed: 2020-02-03.
[57] Kubernetes, "Automating deployment containerized applications," https://kubernetes.io, 2021, [Online; accessed 04-March-2021].
[58] H. M. Gomes, A. Bifet, J. Read, J. P. Barddal, F. Enembreck, B. Pfharinger, G. Holmes, and T. Abdessalem, "Adaptive random forests for evolving data stream classification," *Machine Learning*, vol. 106, no. 9, pp. 1469–1495, 2017.
[59] H. M. Gomes, J. Read, and A. Bifet, "Streaming random patches for evolving data stream classification," in *2019 IEEE International Conference on Data Mining (ICDM)*, 2019, pp. 240–249.
[60] A. Bifet and R. Gavalda, "Adaptive learning from evolving data streams," in *International Symposium on Intelligent Data Analysis*. Springer, 2009, pp. 249–260.

Authorized licensed use limited to: Lebanese American University. Downloaded on August 26,2025 at 19:12:20 UTC from IEEE Xplore. Restrictions apply.